

STM32+BQ76940

BMS 规格说明书

编订日期 2021 年 7 月 06 日

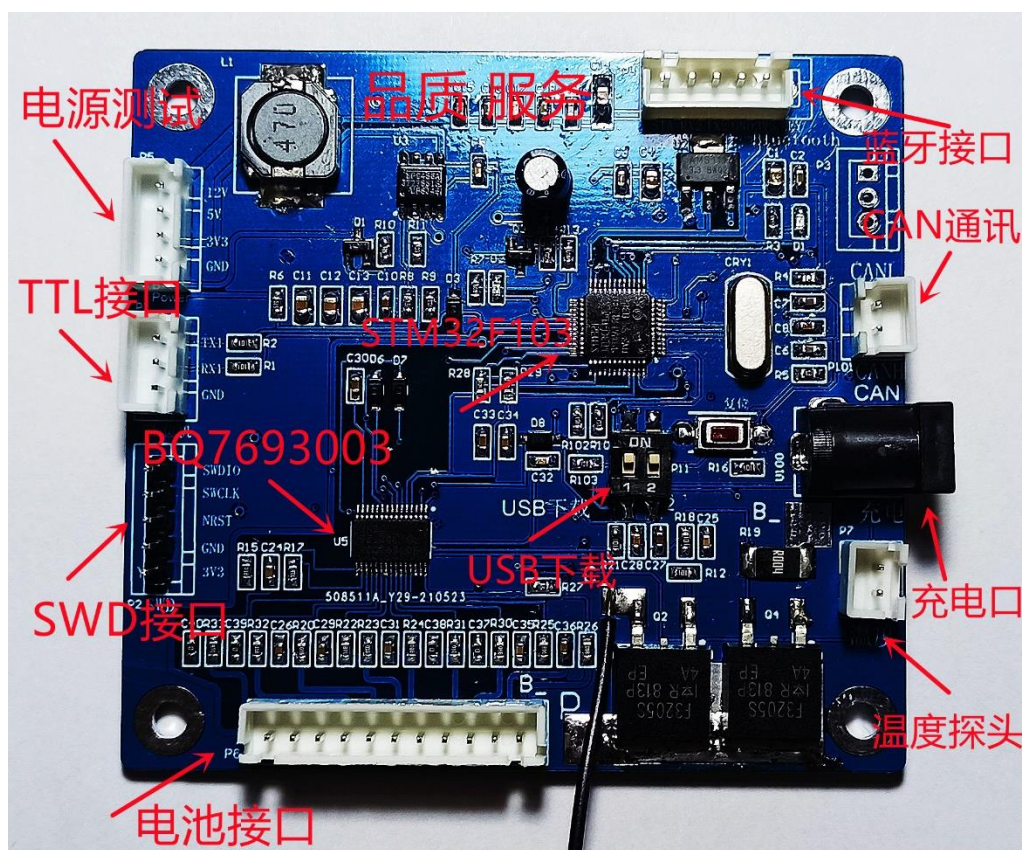
版本号 V1.0

目录

目录

- 系统整体介绍..... 3
 - 系统概述 4
 - 功能描述 5
- 开发板介绍..... 6
- 原理图介绍..... 12
- 上位机介绍..... 12
- 电压、电流、温度采集 13
 - 电压采集 13
 - 电流采集 15
 - 温度采集 16
- SOC 计算..... 17
- 过压、欠压、过流、短路、过温保护 18
 - 过压保护 18
 - 欠压保护 18
 - 过流、短路保护..... 19
 - 过温保护 20
- 通讯协议 21

系统整体介绍



图一：板子正面（BQ76940 与其类似）

系统概述

本系统采用 TI 官方出品的 BQ7694003DBTR 模拟前端 (AFE)，图 1.1 所示为其简要介绍。本 BMS 电池管理系统使用 STM32F103 为主 MCU 使用 IIC 通信与 BQ76940 通信，实现读写 BQ76940 相应寄存器达到读取电池电压，电流，温度等相应数据，然后单片机根据读取到的数据做出相应的判断并做出相应的保护。[点击前往 BQ7693003DBTR DATASHEET 手册](#)

bq769x0 3 节至 15 节串联电池监控系列 用于锂离子和磷酸盐电池应用	
1 介绍	
1.1 特性	
- 模拟前端 (AFE) 监控特性 - 纯数字接口 - 内部模数转换器 (ADC) 测量电池电压、芯片温度和外部热敏电阻 - 一个单独的、内部 ADC 测量电池组电流 (库伦电荷计数器) - 直接支持多达三个热敏电阻 (103AT) - 硬件保护特性 - 放电过流 (OCD) - 放电短路 (SCD) - 过压 (OV) - 欠压 (UV)	- 次级保护器故障检测 - 附加特性 - 集成电池均衡场效应晶体管 (FET) - 充电、放电低侧 NCH FET 驱动器 - 到主机微控制器的警报中断 2.5V 或 3.3V 输出电压稳压器 - 无需 EEPROM 编程 - 高电源电压最大绝对值 (高达 108V) - 简单 I²C™ 兼容接口 (循环冗余校验 (CRC) 选项) - 随机电池连接耐受
1.2 应用范围	
- 轻型电动车辆 (LEV): 电动自行车 (eBike), 电动踏板车 (eScooter), 脚踏电动自行车 (Pedelec) 和踏板辅助自行车 - 电动和园艺工具	- 后备电池和不间断电源 (UPS) 系统 - 无线基站后备系统 12V, 18V, 24V, 36V 和 48V 电池组
1.3 说明	
bq769x0 系列的稳健耐用型模拟前端 (AFE) 器件通常用作针对下一代高功率系统 (例如, 轻型电动车辆、电动工具和不间断电源) 的完整电池组监控和保护解决方案的一部分。bq769x0 在设计时充分考虑了低功耗要求, 不仅可通过使能/禁用集成电路 (IC) 中的子模块来控制整个芯片的电流消耗, 而且还可以利用 SHIP 模式将电池组轻松切换至超低功耗状态。	

器件信息⁽¹⁾

器件型号	封装	封装尺寸 (标称值)
bq76920	薄型小外形尺寸封装 (TSSOP) (20)	6.50mm x 4.40mm
bq76930	TSSOP (30)	7.80mm x 4.40mm
bq76940	TSSOP (44)	11.00mm x 4.40mm

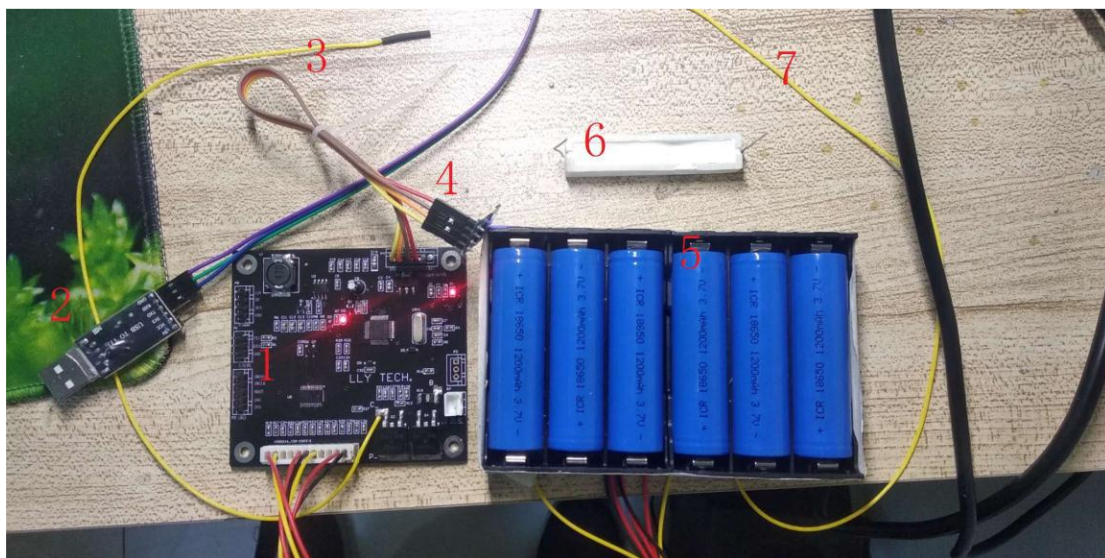
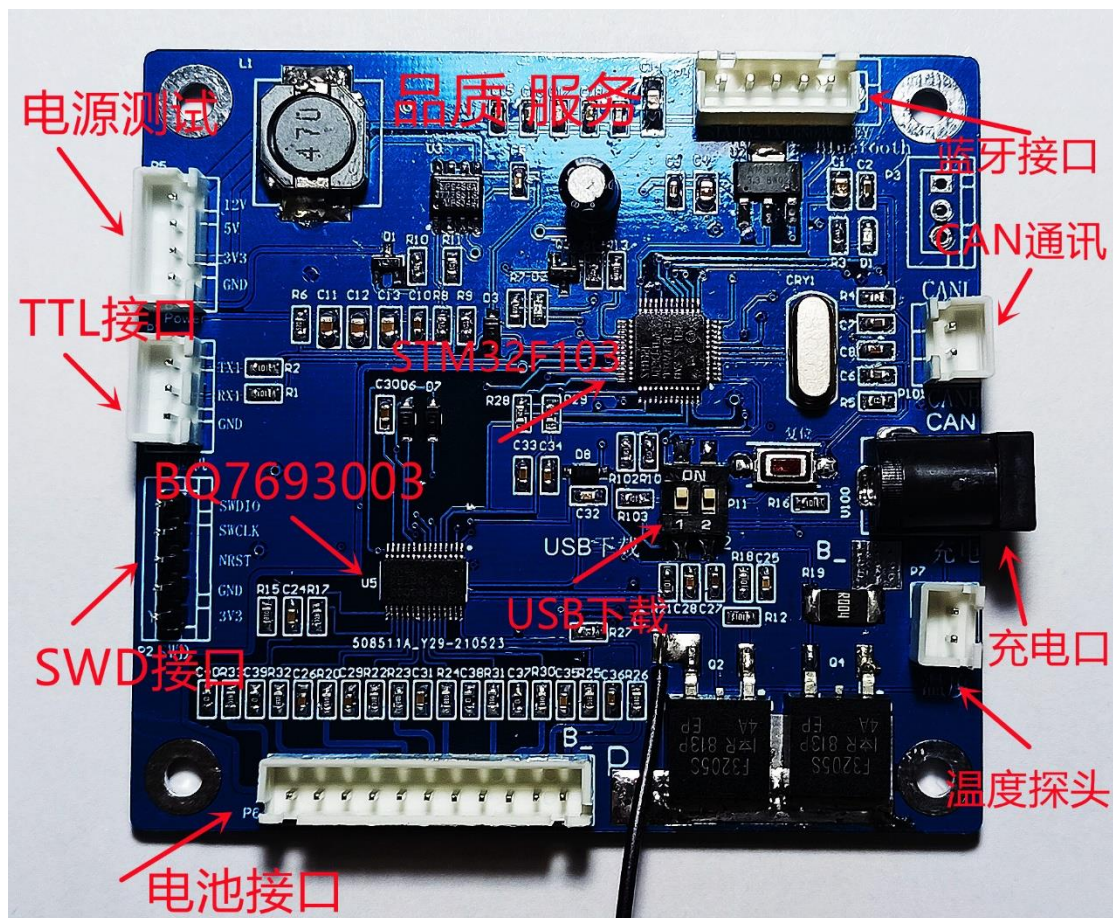
图二: BQ76940 简单介绍

功能描述

本电池管理系统具有如下功能：

- ※ 具有单体电压、总体电压检测，过充、过放告警及保护功能。常温下静态电压采样精度可达 $<20\text{mV}$ 。
- ※ 具有充放电电流检测，充放电过流告警及保护功能。上位机可以显示充放电状态。
- ※ 具有均衡功能，均衡条件程序默认压差大于 50mV ，可设置其它阈值。
- ※ 具有通讯功能，有 TTL, CAN，2 种通讯方式，同时具有蓝牙无线传输功能，通过微信小程序即可查看实时电池信息。
- ※ 具有通过 USB 下载程序功能。免去使用 jlink 下载等比较麻烦的方式。程序下载方法，下载程序需要用到资料包里的 FlyMcu 软件，开发板上的 USB 下载按键“左边拨码”需要拨到下侧“1”的位置，然后开始下载程序（如果不能正常下载可以按下板子复位键或者给板子重新上电再次尝试），下载完毕后将开发板上的 USB 下载按键“左边拨码”需要拨到起初的位置即可。
- ※ 本开发板由于是开发模式，可以通过最大电流最好不要超过 10A ，低功耗模式没有具体去做，感兴趣的朋友可以咨询售后询问具体低功耗逻辑及模式。

开发板介绍



- 1、开发板，使用时请对照原理图了解各接口定义及功能。
- 2、USB 转 TTL 线，连接屏或者上位机的线，使用时注意 TTL 线的接线方式为：RX 接 TX，TX 接 RX，GND 接 GND，连接电脑需要 CH340 驱

动，资料包里有相应的驱动。

3、 USB 下载，使用 TTL 接口既可以连接上位机，也可以接屏，同时还具备程序下载的功能。免去使用 jlink 下载等比较麻烦的方式。程序下载方法，下载程序需要用到资料包里的 FlyMcu 软件，开发板上的 USB 下载按键“左边拨码”需要拨到下侧“1”的位置，然后开始下载程序（如果不能正常下载可以按下板子复位键或者给板子重新上电再次尝试），下载完毕后将开发板上的 USB 下载按键“左边拨码”需要拨到起初的位置即可。

4、 充电口，使用时将充电器插上即可给电池充电。

5、负载连接线，使用时将 3 号线接负载（负载就是白色的大电阻）一端，将 7 号线（电源总正）接另一端即可测出当前流过负载的电流大小。

6、蓝牙模块，接线时注意 3.3V 接板子的 3.3V，GND 接板子的 GND，TX 接板子的 RX，RX 接板子的 TX。

7、电池盒子及 9 节 18650 锂电池，18650 请按照电池盒子上标记的按照串联连接（若一个方向既有+和一，请按照标记的串联连接），往板子上插的时候需要注意方向（方向错了，插不进去）。

8、负载电阻，负载电阻用来测量流过负载的电流大小，使用时将 3 号线接负载（负载就是白色的大电阻）一端，将 7 号线（电源总正）接另一端即可测出当前流过负载的电流大小。

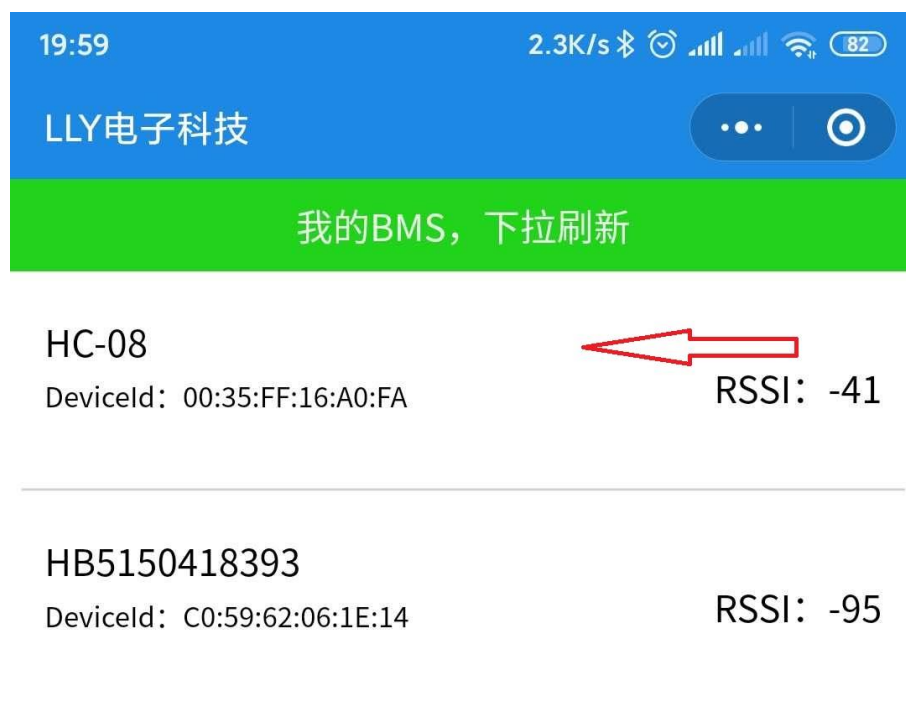
9、电池总正线，也就是 9 节电池的总正，使用时将 3 号线接负载一端，将 8 号线（电源总正）接另一端即可测出当前流过负载的电流

大小。

10、蓝牙小程序 APP，客服会把小程序二维码告诉您，打开微信扫一扫，蓝牙配对成功，即可使用。

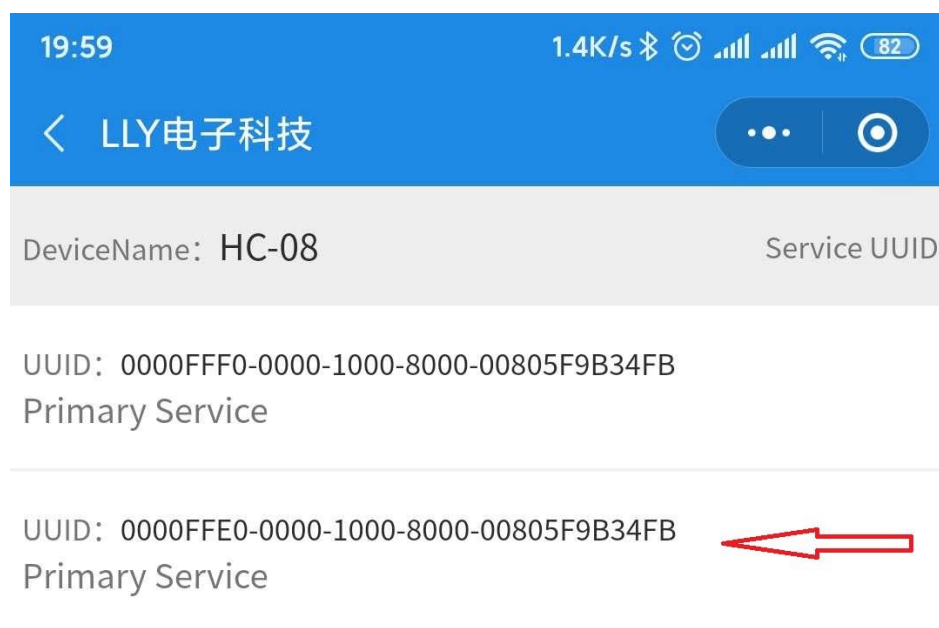


第一步：打开微信“扫一扫”扫描图五二维码会出现如图六所示界面，请选择 HC-08 字样的设备，此设备就是我们蓝牙模块释放的蓝牙；



图六：BQ76930 开发板蓝牙小程序扫描之后界面

第二步：点击 HC-08 之后会出现如图七所示界面，然后选择第二项；



图七：图六的点击之后界面

第三步：第二步操作之后会出现下图所示界面，然后点击进入界

面然后再点击，最终会出现图八所示的锂电池的信息；



图八：图七的点击之后界面



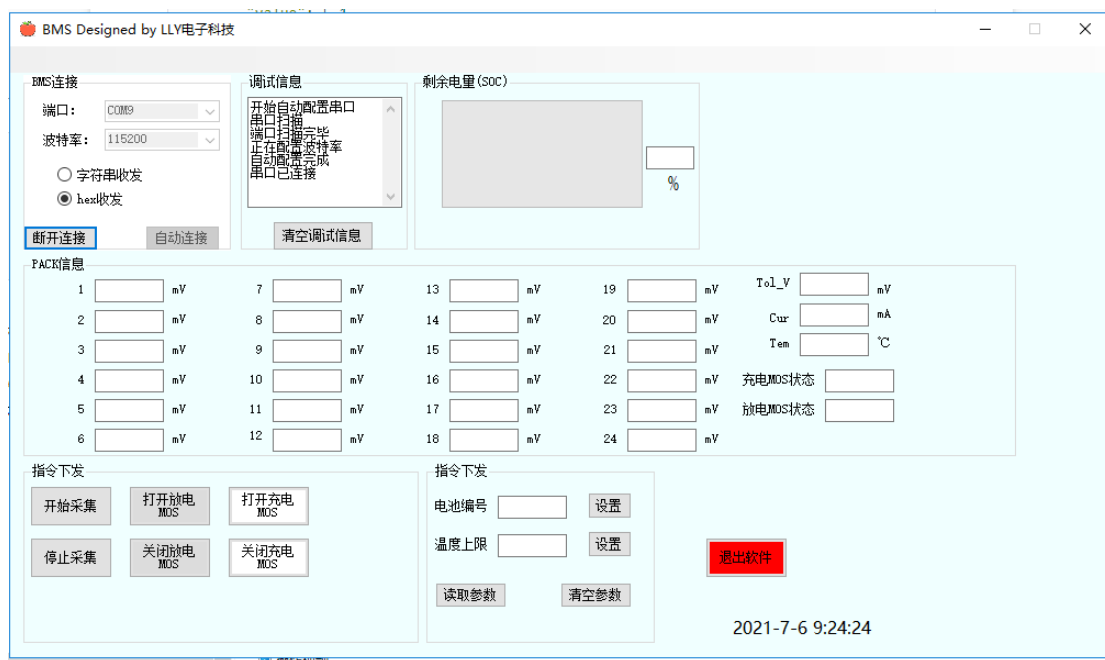
图八：蓝牙微信小程序

原理图介绍

原理图部分由于内容太多，先关介绍请至优酷或者百度网盘观看视频教程，链接如下，密码请找客服索要。

https://v.youku.com/v_show/id_XNDY0Nzk1NzEyMA

上位机介绍



上位机如上所示，上面主要有电池电压，总电压，电流，温度等信息显示，另外有指令的一些控制。上位机采用 C#语言编写，界面如下图所示。使用时将 USB 转 TTL 交叉连接至板子，板子上电后默认充放电 MOS 都是打开状态。

- 1) 点击“开始采集”按钮即可显示电池主要信息
- 2) 点击“停止采集”按钮即可停止板子数据发送给上位机
- 3) 点击“关闭放电 MOS”即可将放电 MOS 关闭，此时将不能实现放电操作，点击“打开放电 MOS”即可将放电 MOS 重新打开

4) 点击“关闭充电 MOS”即可将充电 MOS 关闭，此时将不能实现充电电操作，点击“打开充电 MOS”即可将充电 MOS 重新打开，

电压、电流、温度采集

电压、电流、温度采集是 BQ76940 所具有的基本功能，也是整个管理系统中非常重要的功能，而本系统采集这些数据都是 STM32 用 IIC 通信从 BQ76940 寄存器读出相应的值再做一定的转化即可读出相应的值，只有把数据采集正确才可以对整个 PACK 系统有一个良好的管理。下面就从原理图，源码结合的角度来分别讲解电压、电流、温度的测量是如何实现的。

电压采集

VC1_HI	0x0C	RSVD	RSVD	<13:8>
VC1_LO	0x0D			<7:0>
VC2_HI	0x0E	RSVD	RSVD	<13:8>
VC2_LO	0x0F			<7:0>
VC3_HI	0x10	RSVD	RSVD	<13:8>
VC3_LO	0x11			<7:0>
VC4_HI	0x12	RSVD	RSVD	<13:8>
VC4_LO	0x13			<7:0>
VC5_HI	0x14	RSVD	RSVD	<13:8>
VC5_LO	0x15			<7:0>
VC6_HI ⁽¹⁾	0x16	RSVD	RSVD	<13:8>
VC6_LO ⁽¹⁾	0x17			<7:0>
VC7_HI ⁽¹⁾	0x18	RSVD	RSVD	<13:8>
VC7_LO ⁽¹⁾	0x19			<7:0>
VC8_HI ⁽¹⁾	0x1A	RSVD	RSVD	<13:8>
VC8_LO ⁽¹⁾	0x1B			<7:0>
VC9_HI ⁽¹⁾	0x1C	RSVD	RSVD	<13:8>
VC9_LO ⁽¹⁾	0x1D			<7:0>
VC10_HI ⁽¹⁾	0x1E	RSVD	RSVD	<13:8>
VC10_LO ⁽¹⁾	0x1F			<7:0>

图三 电压采集寄存器

图三所示为电压采集寄存器，只需要读取相应的寄存器位就可以

获得相应的单节电池电压，具体获取电压的方法步骤如下：

- 1、如下所示现将 ADC_EN,TEMP_SEL 置 1，使能 ADC 和温度采集

CELLSEL	MODE	MODE	MODE	MODE	MODE	MODE	MODE	MODE	MODE
SYS_CTRL1	0x04	LOAD PRESENT	RSVD	RSVD	ADC_EN	TEMP_SEL	RSVD	SHUT_A	SHUT_B

```

unsigned char BQ769_INITAdd[11]={SYS_STAT,CELLBAL1,CELLBAL2,SYS_CTRL1,SYS_CTRL2,PROTECT1,PROTECT2,PROTECT3,OV_TRIP,UV_TRIP,CC_CFG};
unsigned char BQ769_INITdata[11]={0xFF, 0x00, 0x00, 0x18, 0x43, 0x00, 0x00, 0x00, 0x00, 0x00, 0x19};
void BQ_I_config(void)
{
    char i;
    for(i=0;i<11;i++)
    {
        delay_ms(50);
        IIC1_write_one_byte_CRC(BQ769_INITAdd[i],BQ769_INITdata[i]);
    }
}

```

图四 STS_CTRL1 寄存器

- 2、读取电池的电压寄存器 0X0C 0X0D,将读取到的寄存器值通过相应的公式待入即可得到电池电压，公式如下：

```

99
00 readbattbuf[1] = IIC1_read_one_byte(0x0c);
01 readbattbuf[0] = IIC1_read_one_byte(0x0d);
02
03 batteryval1= readbattbuf[1];
04 batteryval1= (batteryval1 << 8) | readbattbuf[0];
05 batteryval1= ((batteryval1*GAIN)/1000)+ADC_offset; //单体电压计算公式，第1串
06 Batteryval[0]=batteryval1;

```

图五 读取电压

Batteryval[0]即为第一节电池电压的大小，GAIN 和 ADC_offset 获取方式如下：

```

23 function: void get_offset(void)
24 /***/
25 int ADC_offset, GAIN;
26 float ADC_GAIN = 0;
27
28 void Get_offset(void)
29 {
30     unsigned char gain[2];
31
32     gain[0]=IIC1_read_one_byte(ADCGAIN1); //ADC_GAIN1
33     gain[1]=IIC1_read_one_byte(ADCGAIN2); //ADC_GAIN2
34     ADC_GAIN = ((gain[0]&0x0c)<<1)+((gain[1]&0xe0)>>5); //12uV
35     ADC_offset=IIC1_read_one_byte(ADCOFFSET); //45mV
36     GAIN = 365+ADC_GAIN; //GAIN=377uV
37 }

```

图六 读取 GAIN 、ADC_offset

其它串的电池电压读取方式都是一样，只是寄存器地址不同，可以此类推。

电流采集

CC_HI	0x32	<15:8>
CC_LO	0x33	<7:0>

图七 电流采集寄存器

图四所示为电流采集寄存器，电流的采集方式和上述电压类似，程序里采集电流如下所示：

```
void Get_BQ_Current(void)
{
    unsigned char readCurrentbuf[2];
    unsigned int    Currentbufval;
    unsigned int    Currentval;
    unsigned char Currentbuf[1];
    readCurrentbuf[1]=IIC1_read_one_byte(CC_HI_BYTE);
    readCurrentbuf[0]=IIC1_read_one_byte(CC_LO_BYTE);
    Currentbufval = (readCurrentbuf[1] << 8)+readCurrentbuf[0];

    if( Currentbufval <= 0x7D00 )
    {
        Currentval = (Currentbufval*2.11*3); //单位mV; 按4mΩ计算，单位mA;
        Batteryval[11]=Currentval;
        shang[26]= Currentval>>8;
        shang[27]= Currentval&0X00FF;
        can_buf6[6]=shang[26];
        can_buf6[7]=shang[27];
        if(Currentval>100)
        {
            shang[31]=1;
        }
        else shang[31]=0;
    }
}
```

图八 获取电流值

读取电流寄存器的值通过公式转化为相应的电流数据，计算公式如下图所示：

7.3.1.1.3 16-Bit CC

A 16-bit integrating ADC, commonly referred to as the coulomb counter (CC), provides measurements of accumulated charge across the current sense resistor. The integration period for this reading is 250 ms.

The CC may be operated in one of two modes: ALWAYS ON and 1-SHOT.

- In ALWAYS ON mode, the CC runs at 100%, gathering a fresh reading every 250 ms. The conclusion of each reading sets the CC_READY bit, which toggles the ALERT pin high to inform the microcontroller that a new reading is available. To enable Always On mode, set $[CC_EN] = 1$.
- In 1-SHOT mode, the CC performs a single 250-ms reading, and similarly sets the CC_READY bit when completed. This mode is intended for non-gauging usages, where the host simply desires to check the pack current.

To enable a 1-SHOT reading, ensure $[CC_EN] = 0$ and set $[CC_ONESHOT] = 1$.

The full scale range of the CC is ± 270 mV, with a max recommended input range of ± 200 mV, thus yielding an LSB of approximately 8.44 μ V.

The following equation shows how to convert the 16-bit CC reading into an analog voltage if no board-level calibration is performed:

$$\text{CC Reading (in } \mu\text{V)} = [\text{16-bit 2's Complement Value}] \times (8.44 \mu\text{V/LSB}) \quad (3)$$

计算电流公式

16-Bit CC Result	ADC Result in Decimal	CC Reading (in μ V)
0x0001	1	8.44
0x2710	10000	84,400
0x7D00	32000	270,080
0x8300	-32000	-270,080
0xC350	-15536	-131,123.84
0xFFFF	-1	-8.44

图九 获取电流值公式

温度采集

温度的采集是通过 10K NTC 热敏电阻的值计算得出，10KNTC 热敏电阻的 B 值为 3950，在前述将 TEMP_SEL 使能后就可以获取温度值了，读取温度的寄存器地址为 TS1 TS2 TS3,我们使用的是 TS2,于是我们要读取 TS2 的相应寄存器位读出的值，具体 C 语言的实现过程可参考如下链接

https://blog.csdn.net/qq_41684249/article/details/86743840

TS1_HI	0x2C	RSVD	RSVD	<13:8>
TS1_LO	0x2D			<7:0>
TS2_HI ⁽¹⁾	0x2E	RSVD	RSVD	<13:8>
TS2_LO ⁽¹⁾	0x2F			<7:0>

(1) These registers are only valid for bq76930 and bq76940.

(2) These registers are only valid for bq76940.

32 Detailed Description

版权 © 2013–2015, Texas Instruments Incorporated

[提交文档反馈意见](#)

产品主页链接: [bq76920](#) [bq76930](#) [bq76940](#)



TEXAS
INSTRUMENTS

www.ti.com.cn

bq76920, bq76930, bq76940

ZHCSCE2F – OCTOBER 2013 – REVISED NOVEMBER 2015

Name	Addr	D7	D6	D5	D4	D3	D2	D1	D0
TS3_HI ⁽²⁾	0x30	RSVD	RSVD						
TS3_LO ⁽²⁾	0x31								

```

void Get_BQ1_2_Temp(void)
{
    float Rt=0;
    float Rp=10000;
    float T2=273.15+25;
    float Bx=3380;
    float Ka=273.15;
    unsigned char readTempbuf[2];
    int TempRes;
    unsigned char Tempbuf[1];
    readTempbuf[1]=IIC1_read_one_byte(TS2_HI_BYTE);
    readTempbuf[0]=IIC1_read_one_byte(TS2_LO_BYTE);
    TempRes = (readTempbuf[1] << 8) | readTempbuf[0];
    TempRes = (10000*(TempRes*382/1000))/(3300-(TempRes*382/1000));
    Tempval_2 = 1/(1/T2+(log(TempRes/Rp))/Bx)- Ka + 0.5;
    Batteryval[12] = Tempval_2;
    shang[28]=Batteryval[12];
    温度值
    can_buf7[2]=shang[28];
}

```

图 10 温度寄存器及 C 语言程序

SOC 计算

SOC(State of charge), 即荷电状态, 用来反映电池的剩余容量, 其数值上定义为剩余容量占电池容量的比值, 常用百分数表示。其取值范围为 0~1, 当 SOC=0 时表示电池放电完全, 当 SOC=1 时表示电池完全充满。我们开发板所使用的 SOC 就是电压百分比, 比如单节电芯的电压范围一般为 2800mV 到 4200mV, 我们简单的将其从这个范围划分 0 到 1, 从而做出简单的电量预测。但是此种方法误差较大, 有兴

趣的同学可以参考我们按时积分法去更准确的获取电池剩余电量。

过压、欠压、过流、短路、过温保护

过压保护

过压保护就是当电池电压大于所设定的阈值之后需要切断充电 MOS 开关所做出的的保护，一般来讲，18650 三元锂电池的过压保护阈值为 4200mV，我们单片机通过将采集到的单节电池电压信息与阈值比较，如果单节电池电压大于 4200 的话就关闭充电 MOS 管，程序里具体实现如图 11，imax 为电池组最大电池串电压的值。MOS 管重新开启条件是当电压低于 4000mV 时，充电 MOS 重新打开，恢复正常状态。

```
306      if(imax>OV_VALUE)          //电池最大串电压大于过压保护阈值即关闭充电MOS管；
307      {
308          Only_Close_CHG();
309          OV=1;
310          shang[51]=OV;
311      }
312      if(OV==1)
313      {
314          if(imax<(OV_VALUE-200))//电池最大串电压小于于过压保护阈值-100即打开充电MOS管；
315          {
316              Only_Open_CHG();
317              IIC1_write_one_byte_CRC(SYS_STAT,0xFF); //清除状态
318              OV=0;
319              shang[51]=OV;
320          }
321      }
322  }
```

图 11 过压保护程序

欠压保护

欠压保护就是当电池电压小于所设定的阈值之后需要切断放电 MOS 开关所做出的的保护，一般来讲，18650 三元锂电池的欠压保护阈值为 2800mV，我们单片机通过将采集到的单节电池电压信息与阈

值比较，如果单节电池电压小于 2800 的话就关闭放电 MOS 管，程序里具体实现如图 12，imin 为电池组最小电池串电压的值。MOS 管重新开启条件是当电压高于 2800mV 时，放电 MOS 重新打开，恢复正常状态。

```

23         if(imin<UV_VALUE)//电池最小串电压小于欠压保护阈值即关闭放电MOS管;
24     {
25         Only_Close_DSG();
26         UV=1;
27         shang[52]=UV;
28     }
29     if(UV==1)
30     {
31         if(imin>UV_VALUE) //电池最小串电压大于于欠压保护阈值即打开放电MOS管;
32     {
33         Only_Open_DSG();
34         IIC1_write_one_byte_CRC(SYS_STAT,0xFF); //清除状态
35         UV=0;
36         shang[52]=UV;
37     }
38     }

```

图 12 欠压保护程序

过流、短路保护

过流、短路保护本质上是一样的，都是根据采集到的电流与过流，短路电流的阈值进行比较，只是短路电流阈值肯定要大于过流电流阈值，所以通常是过流电流阈值先起作用，具体程序实现逻辑如下图 13 所示，Batteryval[11]就是采集到的的电流大小。当电流大于 OC_VALUE（过流阈值）就关闭充放电 MOS 管，小于它的时候就重新打开充放电 MOS 管，短路电流比过流大 5A 会触发短路保护，所以是过流保护先起作用。


```

40      if(Batteryval[11]>OC_VALUE)//如果电流大于过流保护阈值，关闭充放电MOS管
41      {
42          Close_DSG_CHG();
43
44          OC=1;
45          shang[53]=OC;
46      }
47      if(OC==1)
48      {
49          if(Batteryval[11]<OC_VALUE)//如果电流小于过流保护阈值，打开充放电MOS管
50          {
51              Open_DSG_CHG();
52              IIC1_write_one_byte_CRC(SYS_STAT,0xFF); //清除状态
53              OC=0;
54              shang[53]=OC;
55          }
56      }
57
58      if(Batteryval[11]>(OC_VALUE+5000))//如果电流大于过流保护阈值+5000ma，关闭充放电MOS管
59      {
60          Close_DSG_CHG();
61
62          DC=1;
63          shang[54]=DC;
64      }
65      if(DC==1)
66      {
67          if(Batteryval[11]<(OC_VALUE+5000))//如果电流小于过流保护阈值+5000ma，打开充放电MOS管
68          {
69              Open_DSG_CHG();
70              IIC1_write_one_byte_CRC(SYS_STAT,0xFF); //清除状态
71              DC=0;
72              shang[54]=DC;
73          }
74      }
75  }

```

图 13 过流、短路程序处理

过温保护

过温保护就是实时温度大于温度上限阈值后，需要关闭充放电 MOS 管，对整个管理系统做出保护，具体程序实现如下图 14 所示，当温度大于阈值关闭充放电 MOS 管，当温度小于阈值就打开充放电 MOS 管。

```

379
380      if(Batteryval[12]>AT24CXX_ReadOneByte(6))//如果温度大于温度保护阈值，关闭充放电MOS管
381      {
382          Close_DSG_CHG();
383
384          OT=1;
385          shang[55]=OT;
386      }
387      if(OT==1)
388      {
389          if(Batteryval[12]<AT24CXX_ReadOneByte(6))//如果温度小于值，打开充放电MOS管
390          {
391              Open_DSG_CHG();
392              IIC1_write_one_byte_CRC(SYS_STAT,0xFF); //清除状态
393              OT=0;
394              shang[55]=OT;
395          }
396      }
397  }

```

图 14 温度阈值保护逻辑

通讯协议

本管理系统支持 TTL 通讯， CAN 通讯，蓝牙无线通讯，具体协议详见通信协议文档。

致谢

凡在本店购买任何产品均提供免费技术支持，如果需要任何配件，也可以联系本店客服，希望多多光临本店，多谢朋友们的支持，祝各位朋友事业有成！

淘宝店地址：

<https://shop126255972.taobao.com/?spm=2013.1.1000126.d21.39201cb43umum2>

本店铺售后 QQ 为：[3526288484](https://www.qq.com/3526288484)，TEL/微信：18611379203，欢迎添加和随时咨询！